

Getting Unstuck, Staying On Task: Designing Local Small Language Models to Support Student Learning

ANONYMOUS AUTHOR(S)

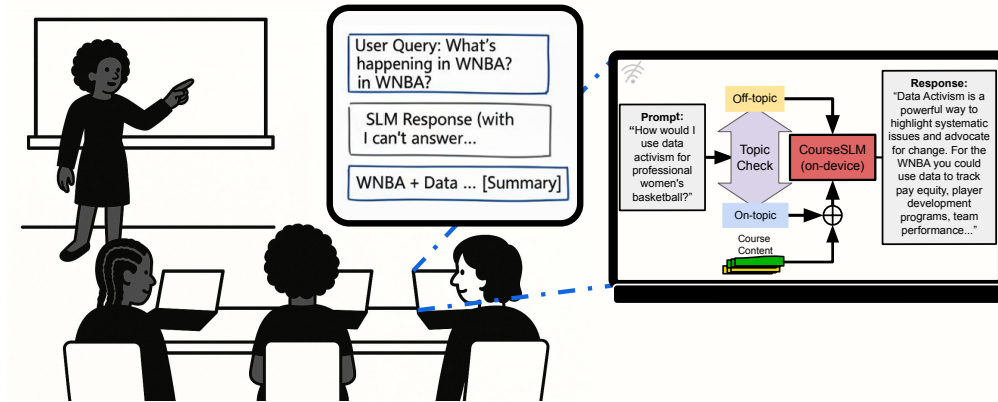


Fig. 1. Students interact with CourseSLM while in a class. CourseSLM (a local, small, fine-tuned language model) captures students' prompts and guides them back to the course content, helping them stay focused.

Large-language models (LLMs) are increasingly used in computing education, but risk distracting learners, fostering overreliance, and compromising privacy. We introduce CourseSLM, a small language model (SLM) and chatbot interface tailored to support course-specific learning on student laptops. Our SLM maintains performance by providing constrained, lesson-specific assistance and leveraging pre-prompting, retrieval-augmented generation on course material, and fine-tuning on example question-response pairs. CourseSLM discourages distractions through a student-designed interface and classifier that redirects off-task queries to course-relevant interactions. We evaluated CourseSLM in three classroom conditions: our interface with SLM, our interface with a general-purpose LLM, and a general-purpose chatbot. Our interface improved task focus, and our SLM performed comparably to cloud-based LLMs in helping students overcome technical barriers and maintain learning momentum, though both occasionally produced lengthy or jargon-filled responses. We discuss user experiences and how these findings can inform future local, fine-tunable AI tools that promote engagement, learning, and data stewardship.

CCS Concepts: • **Do Not Use This Code** → Generate the Correct Terms for Your Paper.

Additional Key Words and Phrases: Local-first AI, Data sovereignty, Programming education, Learning analytics, Prompting, Focus, Stuckness, On-device LLMs, Scaffolded UI

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

ACM Reference Format:

Anonymous Author(s). 2018. Getting Unstuck, Staying On Task: Designing Local Small Language Models to Support Student Learning. In *Proceedings of Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. ACM, New York, NY, USA, 30 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large language models (LLMs) have emerged as transformative tools for computing education [63]. They offer students natural language explanations, can support self-driven learning, as well as automated code generation and debugging [14, 48]. In particular, these capabilities offer exciting potential to reduce persistent inequities in computing education for underrepresented minority (URM) students. AI tools could generate culturally-affirming explanations [22, 36], support near-peer mentorship by extending the capacity of limited, qualified instructors [30, 31, 55], and provide interactive feedback for learning outside the classroom.

Despite their potential, general-purpose chatbots also introduce multiple new pedagogical risks. By generating direct solutions, students bypass productive struggle, which is essential for developing critical thinking and computational problem-solving abilities [18, 32, 34]. Similarly, hallucinated responses can be harmful for novice students learning new subjects [21, 67]. General-purpose LLMs also readily accept off-topic prompts, providing distractions that reduce time on task which harms learning momentum and knowledge retention [18, 62]. These primarily cloud-based services also present substantial barriers for use in low-resource settings due to high subscription costs and inconsistent internet access [47]. Further, routing student queries with personal information or vulnerable struggles with course material to cloud providers also raises substantial concerns about ownership, consent, and misuse especially in historically marginalized communities [8, 9, 12, 31, 51].

These challenges highlight a critical need for educational AI systems that align with classroom realities while supporting student learning, preserving student privacy, and supporting equitable access [56]. To address this need we develop CourseSLM, a fine-tuned small language model (SLM) and chatbot interface tailored to support course-specific learning and run locally on student laptops. Our CourseSLM prototype is developed in collaboration with a STEM education community organization for use in a data activism course targeting URM K-12 students.

CourseSLM's custom chat interface shown in Fig. 1 is designed to minimize distractions, and includes a classifier that detects off-task queries and uses a prompt wrapper to redirect students to course-relevant interactions. The backend of CourseSLM is a SLM fine-tuned on curated question-response pairs that uses retrieval-augmented generation from course slides to provide course-relevant responses. CourseSLM is designed to run using Apple's MLX framework [2, 24] which enables real-time on-device execution.

We leverage this tool across three user study sessions with 19 total participants (ages 15–36, mean = 20) in a classroom setting to answer the following research questions:

RQ1: How do AI tools like CourseSLM and ChatGPT-4o support student task focus?

RQ2: How do AI tools like CourseSLM and ChatGPT-4o help students overcome technical barriers?

RQ3: How do AI tools like CourseSLM and ChatGPT-4o help students sustain learning momentum?

RQ4: How do AI tools like CourseSLM and ChatGPT-4o affect task performance and completion?

Our quantitative and qualitative results show that the CourseSLM architecture can identify off-topic queries and the backend SLM can redirect students back to course material. We also find that while it achieves its design objective of not providing direct solutions, this frustrates some users with AI experience who expected to skip learning and

comprehension and receive complete code solutions. This key finding highlights that tools must go beyond providing outputs that encourage learning, and must more broadly reframe students' expectations of AI assistants.

In summary, this work contributes to (1) a novel educational AI tool designed with features to support focused, course-specific learning on student laptops, and (2) we use CourseSLM and ChatGPT-4o to study how AI tools performs in classroom settings. We reflect on the development process of CourseSLM and discuss how our findings can inform future AI education tools. Specifically, we find the need for tools to promote a broader mindset shift in how students use AI that promotes both student learning and advances educational equity.

2 Related Work

2.1 AI Tools in Computing Education

Before generative AI, intelligent programming tutors (IPTs) were the primary AI tools for supporting computer science learning outcomes. These tools were often scoped to specific universities, courses, and concepts such as recursion or object-oriented programming (OOP) [10]. IPTs and auto-grading systems such as ILMDA were used for practical learning and task verification [53]. While they provided adaptive feedback and corrected syntax errors, they relied on generalized models of student behavior and lacked integration with broader course materials to support conceptual growth [10]. In contrast to these systems, which often rely on cloud infrastructure and broad general-purpose LLMs, our study investigates a locally-run, course-specific model designed with community-representative instructors. We contribute empirical evidence of how such a model may support novice programmers in real classroom use.

In a systematic literature review of AI in computer science education between 2003 and 2023, Zhu & Zhang identified pedagogical theories such as personalized learning, problem-solving, self-regulated learning, gamification, inquiry-based learning, and collaborative learning that guided AI tool design [9, 70]. The review also raised concerns about opaque learning metrics and called for more qualitative studies to explore how AI influences the learning process. Generative AI supports some of these pedagogies, and has been shown to lower the threshold for novice learners to scaffold executable code and empower more diverse groups to access programming skills [1, 14, 20]. LLMs can generate personalized materials, synthesize insights from noisy data [15], and support non-academic learners [71]. They have been deployed for debugging, hint generation [7, 29], and in course environments such as Iris [4] and 61A-Bot [65]. Although students often value timely support, various studies have also cautioned that LLMs can reduce engagement, obscure learning obstacles, and disrupt rather than sustain productive learning trajectories [19, 34, 62]. Building on this body of research, we explore the use of a locally-run, course-specific model similar to the design of IPT systems but with the natural language, reasoning and information retrieval capabilities of generative AI. We further tailor our interface design to specifically support student focus and engagement.

2.2 Concerns with Generative AI in Computer Science Education

2.2.1 Overreliance and Impacts on Learning Outcomes. Prior work highlight how general-purpose LLMs can encourage overreliance, providing direct answers to students' questions instead of scaffolding problem-solving skills [32, 38]. This echoes Prather et al.'s "shepherding" and "drifting" prompting patterns, which highlight how students either push tools for complete solutions or abandon them when their distrust grows [45]. We extend these findings by examining how overreliance manifested in classroom use, we discuss our findings in Section 5 below.

2.2.2 Preserving Student Data Privacy. Cloud-based LLMs raise well-documented concerns about data privacy, ownership, and sovereignty in education [12, 57]. These issues are acute in classrooms, where prompts may contain personal

identifiers or FERPA-protected content. Few studies have examined how educational AI systems might explicitly address these broader values, highlighting a gap in the literature. To address these concerns, we deployed CourseSLM on Apple’s MLX framework [2], running fully on-device to keep data local despite computing constraints that hinder performance and often limit accessibility. We also examine how students perceive and trust AI responses in this setting.

2.2.3 Equity-Focused and Human-Centered Design. Recent work in computing education and AI has increasingly researched questions of equity, cultural responsiveness, and student agency. For example, Solyst et al. investigate young people’s critical AI literacies, identity, and fairness in computing education [54] and Morales-Navarro et al. examine how youth make sense of machine learning through conceptual, creative, and ethical lenses [40]. These studies highlight the importance of culturally grounded curricula, participatory design, and youth-led sensemaking, underscoring how neglecting learners’ identities, experiences, and cultural values risks reinforcing existing inequities [70].

Building on this foundation, we designed CourseSLM in collaboration with community-representative undergraduate instructors and fine-tuned the model using culturally responsive curricula [59–61]. Our approach embeds these values throughout the design process, aligning model behavior with the pedagogical values of underrepresented student groups, and positioning CourseSLM as an alternative to general-purpose LLMs. By doing so, we prioritize equity alongside technical performance, and evaluate how these equity-centered design choices shape student engagement in classroom settings.

2.3 “Just-Enough Help” in AI Design

Recent work emphasizes that AI should provide “just-enough help,” supporting productive struggle without replacing human reasoning [3, 13, 66]. Still, most prior tutors and LLM-powered assistants have emphasized correctness and efficiency over student agency [10, 11, 53]. CourseSLM takes a different approach, demonstrating that “just-enough help” can be operationalized through constrained outputs and a refocusing architecture that nudges students back to course content, with the goal of maintaining engagement and learning momentum. This is unlike general-purpose LLMs that often provide full solutions or allow off-task exploration [11, 66], extending theoretical calls for scaffolding into demonstrated classroom practice.

3 Design of CourseSLM

3.1 Design Goals

CourseSLM can be divided into three distinct components: the user interface, backend system architecture, and the fine-tuned SLM. The backend processes each user prompt by routing it through a binary classifier, a prompt wrapper, and a retrieval-augmented generation (RAG) module for on-topic inputs, before passing it to the fine-tuned SLM to generate a response (see Fig. 2). The development team consisted of undergraduate research interns who gained experience in language model design, user interface development, and computing curriculum design. These interns also served as active instructors in the partner community-based organization, ensuring that the tool reflected both technical training and classroom practice. CourseSLM’s design goals were informed by the knowledge of its developers, who served as student instructors for K-12 learners from underrepresented backgrounds with little to no programming experience.

The undergraduate research intern team further refined these goals through consultations with undergraduate STEM peers, graduate student mentors, and faculty advisors. The resulting design goals (DGs) from the developer team’s reflexive inquiry is as follows:

Manuscript submitted to ACM

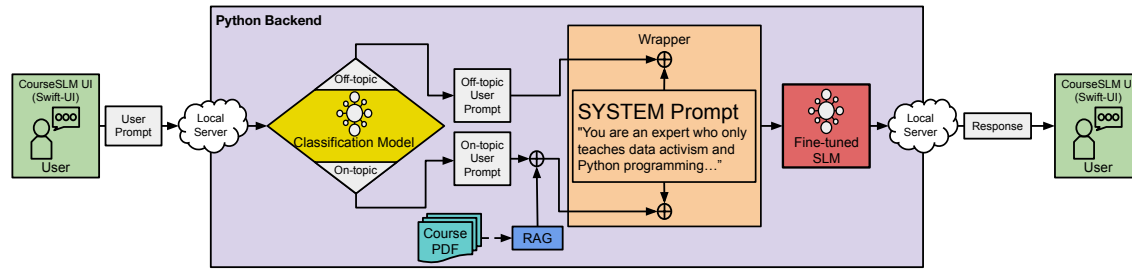


Fig. 2. Systems design behind the CourseSLM app. Starts with the user prompt that is then processed by the classifier directing the prompt into two channels to either get more context through RAG and reply to the student in the context of the activity or kindly guide the student back on topic in case they strayed.

DG1: Provide stepwise support for programming questions instead of direct solutions. In classroom settings, instructors often noticed students would quickly generate code for assignments without a deeper understanding of the intended learning outcomes. The lack of deeper understanding often manifested in overreliance on the AI for debugging and struggles with articulating their technical issues in their prompts. To address this, the SLM within CourseSLM offers stepwise guidance, starting with subtle nudges and escalating to hints and targeted examples. This approach aims to preserve intellectual agency and encourage problem-solving resilience, without offering complete code solutions.

DG2: Maintain student focus and reduce distractions within the user experience. Our instructors observed that students often used generative AI tools to pursue personal interests unrelated to classroom tasks. Because online platforms are already common avenues for disengagement and are sometimes restricted by campus networks, we chose a locally run framework to avoid introducing another potential distraction or access barrier. Within this framework, prompts are routed through a binary classifier that redirects off-topic queries back to course-relevant discussion, helping keep interactions aligned with the learning task. To further support focus, our UX student interns conducted comparative analyses of existing AI tool interfaces and iterated on design choices to develop a more focus-friendly experience.

DG3: Preserve student data privacy with local computing infrastructure. Our instructors noted that students often shared not only their technical struggles but also personal frustrations and insecurities when interacting with AI tools [18]. Protecting this sensitive data was a core design priority. Rather than relying on cloud-based systems that transmit student inputs to external servers, we built CourseSLM to run fully on local devices. By leveraging Apple’s MLX framework [24], the system executes inference directly on student laptops, ensuring that prompts and outputs remain on-device. This design both reduces reliance on large, opaque models and addresses concerns about data sovereignty. Running locally also improves accessibility by enabling deployment in schools with limited infrastructure, where bandwidth and cloud access may be unreliable. In this way, the system design preserves privacy, supports equity in access, and provides a trustworthy foundation for classroom use.

3.2 Technical Architecture

An overview of CourseSLM’s technical architecture is shown in Fig. 2. User queries are first classified as on- or off-topic. The user’s raw inputs are then augmented with system prompts and course-specific content to produce the final response. We discuss each block in detail below.

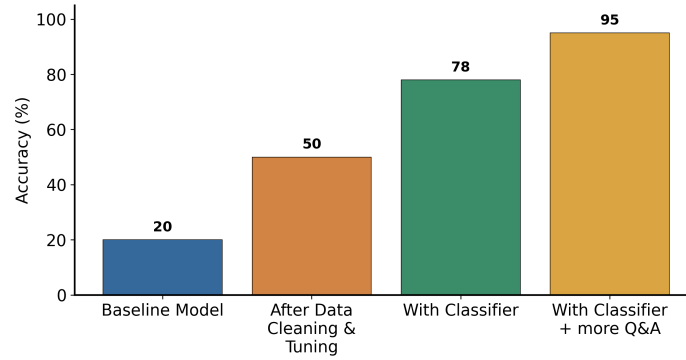


Fig. 3. This bar graph displays how well various architectures performed classification for user prompts as on- or off- task. The best set-up was our classifier after it had been improved with more training data.

3.2.1 Binary Classifier. We fine-tuned a distilled version of the BERT base model [52] to classify user prompts as either “on-topic” or “off-topic.” The training dataset included over 800 prompts drawn from in-class chat logs with K-12 students from ORG’s lessons, collected with informed consent [18]. On-topic examples reflected programming terminology and “how-to” questions, while off-topic examples often reflected pop culture and gaming discussions. To improve classification accuracy, especially for novice queries, we augmented the base dataset with AI-generated questions and labels modeled on the baseline data (see Fig. 3).

3.2.2 Retrieval-Augmented Generation (RAG). When prompts are classified as on-topic, CourseSLM uses retrieval-augmented generation to ground responses in community-relevant curricula. We converted the ORG’s adapted Data Activism lesson plans and coding assignment into PDFs [59–61], segmented them into overlapping text chunks of 800 characters with a 50-character overlap above and below each chunk, then embedded them using a sentence-transformers model [16]. A similarity search retrieves the most relevant text chunk, which is then passed to the prompt wrapper. This design ensures that responses stay grounded in course material, consistent with prior findings that RAG improves contextual relevance [15].

3.2.3 Prompt Wrapper. Using the Jinja templating language, we implemented a structured prompt wrapper derived from the CHATJINJA script generated during fine-tuning, which organizes the system prompt (see Fig. 2) and user input to support consistent and effective prompt engineering [26, 27, 46]. Each prompt was constructed from three components: the system prompt, retrieved curriculum text chunk, and the original user query. This wrapper applies prompting techniques shown to improve model reliability such as prompt hygiene, input sanitization, and role separation [14, 69]. On-topic prompts are contextualized with relevant curriculum material from the RAG, while off-topic prompts are flagged so the model can redirect students and encourage course-relevant engagement.

3.2.4 Fine-Tuned Small Language Model. The core of CourseSLM is a fine-tuned version of Qwen2’s 1.5B parameter model sourced from HuggingFace [64]. Undergraduate instructors fine-tuned the model with ideal examples of question–answer pairs from the Data Activism course. The dataset was expanded on with on-topic questions paired with stepwise guidance responses rather than direct solutions, as well as off-topic questions paired with examples of how to redirect students back toward course-relevant topics. The model escalates support in three tiers: a nudge, a targeted hint, and a concise example. To study interaction cycles, we designed CourseSLM to log prompts, classifications, model

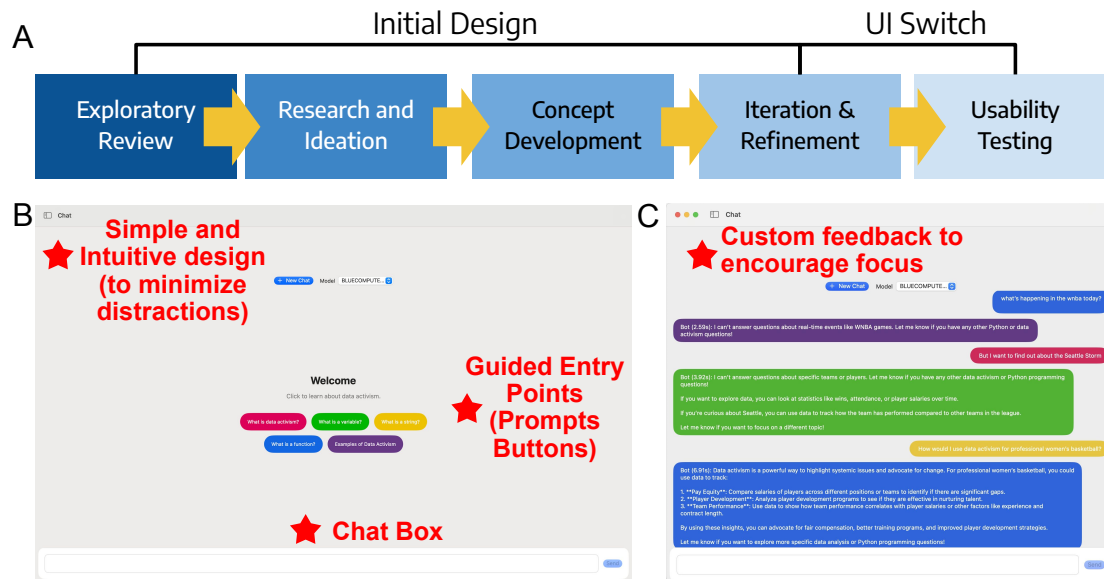


Fig. 4. A) Our iterative design process. B) The main user interface. C) A UI example showing replies to an off-topic question.

outputs, and response times locally using CSV storage for later analysis. Importantly, by leveraging Apple's MLX framework [24], all data and computation remained on-device, preserving privacy while enabling dynamic memory for iterative conversations.

3.3 CourseSLM User Interface

The CourseSLM interface adopts an iMessage-style chat to minimize the learning curve and reduce cognitive load, keeping students focused on course tasks. The design, shown in Fig. 4 follows familiar chatbot conventions and native system themes. Starter prompts (e.g., "What is data activism?" or "Who are important figures in data activism?") orient students and signal that the model is aware of the Data Activism activity at hand, while leaving space for free-form queries [43]. To reduce distraction, controls are deliberately minimal, with no image generation, side panels, or extraneous widgets, with all session data stored and viewable locally to preserve privacy.

Our first prototypes used a colorful, dynamic palette drawn from ORG's brand colors, reflecting research that vibrant colors can enhance engagement [37]. In early pilot studies, students described the design as visually friendly, but some found the random color variation distracting. To address this, we streamlined the palette and reduced variance while retaining a welcoming tone.

Pilot User testing also informed refinements to improve wayfinding and momentum. We added a lightweight first-run banner with actionable suggestions to reduce initial ambiguity, consolidated actions to simplify information architecture, and introduced subtle separators (e.g., labels and timestamps) to improve legibility. Together, these adjustments preserve a low-friction chat experience while reducing early confusion and supporting sustained engagement.

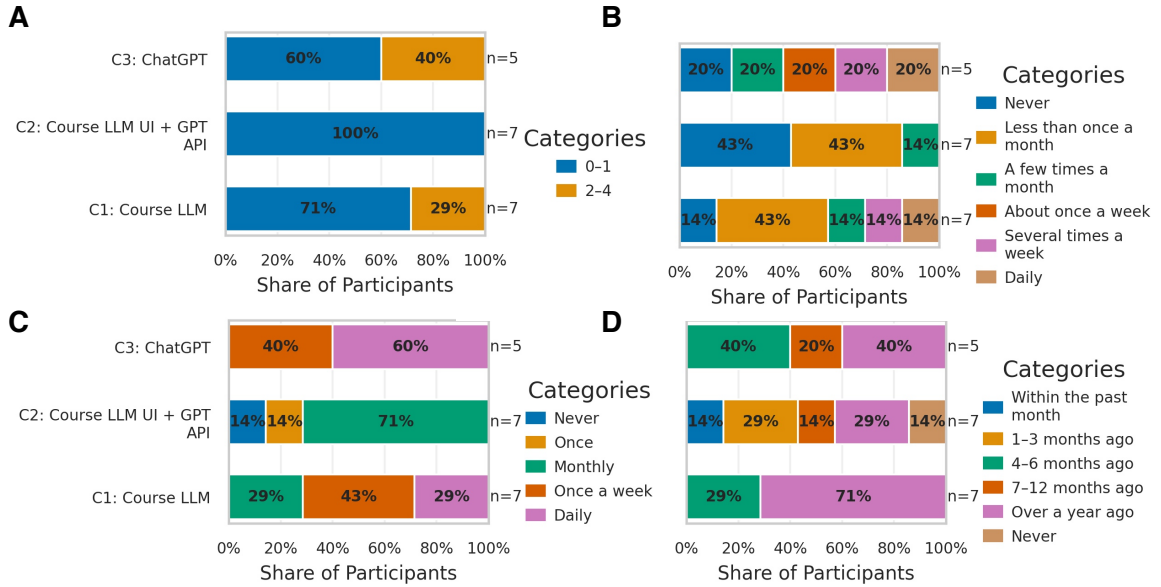


Fig. 5. **A) Years of Coding Experience:** Most participants report ≤ 1 year of coding experience. **B) Frequency of Writing Code:** C1–C2 have lighter writing-code frequency, with many reporting “less than once a month” or “never.” **C) LLM Use Frequency:** LLM use is most frequent in C3 (40% weekly, 60% daily) and primarily monthly in C2 (71%). **D) Start Time of LLM Use:** Adoption timing is mostly >1 year ago in C1 (71%).

4 Methods

4.1 Recruitment and Participant Demographics

We recruited 19 participants (9 women, 10 men, mean age = 20) through a university campus and community affinity groups, with intentional outreach to underrepresented students (Black, Hispanic/Latino, and Indigenous). Participants were strategically placed across the three study conditions by coding experience and prior AI use to ensure balanced skill levels across testing conditions (see Table 1 and Fig. 5). Participants included both secondary school students from prior ORG outreach programs and university-level learners. The study was approved by our Institutional Review Board (IRB), and all recruitment, incentives, and demographic collection followed IRB protocols. Demographic questions were optional, and all data were anonymized using assigned pseudonyms to protect participant identities.

4.1.1 User-Study Protocol. Studies were conducted in a small classroom at our institution. Standalone desks were arranged to face a lectern in which the instructing facilitator was positioned and projector screen displayed a slideshow that explained the study and given curriculum (see Fig. 1). Before each session, participants completed a pre-survey to record their level of programming and AI tool experience. The participants were also provided a physical copy of the consent form for the participants and guardians (if participants were under 18 years old) to review and sign. The study facilitator conducted an icebreaker to create a comfortable environment, mimicking a classroom setting and fostering rapport between researchers and participants.

Study participants were then given an introduction to the research team and informed that the study facilitator, who was positioned as the acting instructor, was a constant resource available for students throughout their progress on the

tasks. Participants were informed that non-facilitating researchers would walk around and take notes on their progress and activities. They were instructed to ignore them and treat the study as a basic classroom.

Participants were instructed to complete a Data Activism assignment focused on CO_2 emissions and carbon footprint trends by country and year, developed by ORG in a Google Colab notebook. The assignment included various programming concepts such as variables, conditional statements, for loops, functions, print and import statements, and data frames. Each cell in the notebook contained sub-tasks that students could progressively work through to complete the assignment. The notebook also provided precise definitions and code examples for each concept.

Students were instructed to consult the AI tool for technical and conceptual support. Students were also offered a slideshow which included precise explanations and code examples informed by undergraduate-level introductory computer science curriculum. If they were unable to overcome their learning barrier within 5 minutes, they were encouraged to ask the instructor for help. The Google Gemini interface integrated into Google Colab was disabled to encourage participants to use the given study resources. A timer for 45 minutes was set and displayed at the front of the classroom during each study.

Once the 45 minutes elapsed, the instructor facilitator led a short debrief discussion in which participants gave overall impressions of the experience, obstacles they encountered using the AI tool, and aspects of the experience they enjoyed. After the debrief session, participants were given the opportunity to take part in an exit interview. Those who chose to participate engaged in a more detailed semi-structured interview with the observing researchers, lasting approximately 15 minutes. After completing the study, all participants were compensated with a \$25 Amazon gift card.

4.2 Study Conditions

The research team aimed to maintain a consistent research environment across conditions. Participants were under an identical study protocol aside from the AI tool they used. Fig. 6 shows the unique architecture of each condition.

- **C1: CourseSLM architecture + SLM:** Students had access to our complete tool, CourseSLM with our fine-tuned model acting as the AI backend. It included all of the components mentioned within system design.
- **C2: CourseSLM architecture + LLM (ChatGPT-4o):** Students had access to the CourseSLM interface with queries redirected to ChatGPT-4o mini [28] as the AI backend. This condition compares the performance of our local SLM versus cloud LLMs while maintaining other features such as on-topic checks, RAG, and prompt wrappers.
- **C3: ChatGPT-4o Interface + model (baseline):** Students had access to unmodified ChatGPT-4o, with no intermediary user interface.¹

4.3 Data Collection & Analysis

4.3.1 Quantitative Data. To address RQ1 (Student Task Focus), we analyzed the chat logs from all conditions. Each user prompt was programmatically classified as either “on-topic” or “off-topic” using our binary classifier and manually labeled by the study team to determine ground truth. We then calculated and compared the percentage of off-topic prompts across the three study conditions to measure the effectiveness of each tool in maintaining student focus.

To address RQ2 (Overcoming Technical Barriers) and RQ3 (Sustaining Learning Momentum), we measured the time spent and rate of progress on the lesson’s subtasks (each cell in the lesson’s Google Colab notebook) before a participant “stalled.” A participant was considered “stalled” if the time they spent on a given task exceeded a dynamic

¹ChatGPT-5 was released shortly before our sessions, we were unable to access ChatGPT-4o mini through the chat interface hence the design choice.

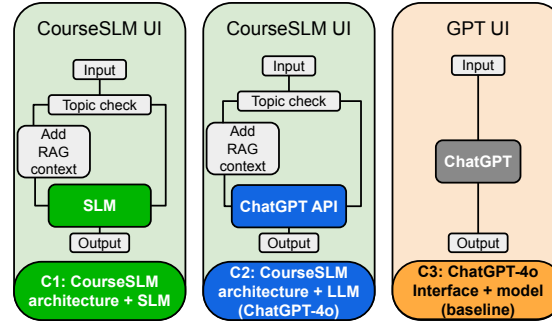


Fig. 6. Diagrams showing the input-to-output flow for tools in each condition. Condition 1 (left) evaluates the complete CourseSLM system combining the UI and SLM model backend. Condition 2 (center) evaluates the CourseSLM UI only with a ChatGPT backend. Condition 3 (right) evaluates unmodified ChatGPT.

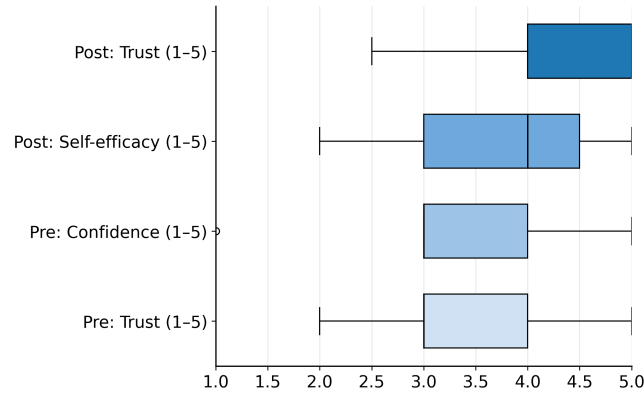


Fig. 7. **Pre/Post Self-Report Outcomes (1-5)**. This figure is a box plot showing pre- and post-study self-reported outcomes on a 1-5 scale. **Pre: Trust in LLM Output Accuracy (1-5)** — Baseline trust in the accuracy of LLM outputs. **Pre: Confidence to Finish Study Programming Tasks (1-5)** — Agreement with “I am confident in my ability to finish the study’s programming tasks,” mapped from *Strongly disagree* (1) to *Strongly agree* (5). **Post: Self-efficacy for Similar Tasks (1-5)** — After the activity, confidence in doing similar tasks in the future. **Post: Trust in LLM Output Accuracy (1-5)** — Post-study trust in the accuracy of LLM outputs. The chart shows a general increase in both trust and self-efficacy/confidence from pre- to post-study.

threshold. This threshold was defined as the 90th percentile of completion times for all successfully completed tasks, but was adjusted based on a participant’s self-reported prior coding experience (0–1 years vs. 2–4 years) to create a more equitable measure of whether they were “stuck” on a task. A higher number of cells completed before stalling indicates better momentum and success in overcoming barriers. The results of this analysis are detailed in Section 5.

In alignment with RQ3, we measured the ability of each tool to sustain a smooth learning experience. We analyzed session logs to determine the average model response time and administered pre- and post-study surveys. These surveys captured participants’ self-reported confidence in being able to perform similar programming activities in the future. This outcome, detailed in Fig. 7, serves as a key quantitative indicator for sustained learning momentum.

Finally, to address RQ4 (Task Performance), we calculated the overall task completion percentage for each participant. This was determined by dividing the number of notebook exercises a participant successfully completed by the total

number of exercises in the assignment. We then aggregated these percentages to compare the average task performance across the three conditions.

4.3.2 Qualitative Data. We conducted semi-structured exit interviews - an HCI research method used to understand participants' experiences and desires [42], which provided insight into RQs 1, 2, and 3. Our questions centered on student focus, learning barriers, learning momentum, perceived trust, and user experience throughout the study. Exit interviews were conducted with 3 to 4 participants chosen to represent the full spectrum of users including those who struggled and excelled on the assigned task or used the tool in unexpected ways.

The study facilitator asked temperature-gauging questions, first inquiring about participants' experience using the tool then further probed based on feedback. This gave us insight into the tool's helpfulness and users' sense of control, trust, and their interest in using a tool like this in the future.

To gain insight into RQ3, we analyzed prompt logs stored locally on the device. Prompt logs contain time stamps, and relative interaction data in which we were able to examine the amount of time spent on tasks, prompts, or idling. We were able to measure the number of prompts per minute and the time between each prompt and re-prompt. This prompt-log instrumentation follows prior LLM studies that quantify task-prompt dynamics in applied settings [14].

Notetakers kept track of participants who used a mixture of strategies to assist with task completion. This includes instructor support alongside the AI resources we provided. Notetakers also recorded learning barriers that took more than 10 minutes to overcome. They recorded protocol deviations and notable quotes with timestamps, all in alignment with RQ2.

4.3.3 Data Analysis. We employed techniques from reflexive thematic analysis [6] to inductively and deductively [17] code exit interview transcripts and chat logs. The research team conducted multiple rounds of coding until reaching thematic saturation. As we deductively code observations from facilitator notes and participant screen recordings, we further developed our understanding of our deductive themes by inductively surfacing sub-themes. Throughout the qualitative analysis process, the research team tasked with qualitative analysis met regularly to build consensus through reflexive and embodied discussions of participants (see Appendix B for the full codebook). First, two members of the research team utilized Dovetail - qualitative data analysis software to analyze study materials such as screen recordings, exit interview transcripts and videos, facilitator observation notes, and AI tool chat logs for analysis. The initial analysis team conducted inductive open coding on the exit interview transcripts to establish initial themes of interests that can be mapped on the research questions.

After this analysis, the two team members shared their initial findings with the rest of the research team to spur discussions of emerging themes and iterate on research themes. After this initial round of coding and discussion, a team of four research members thematically coded participant chat logs deductively using the themes of the research questions. AI tools like Gemini and ChatGPT were leveraged to aid in the development of the deductive codebook, divide chat logs into conversation episodes (prompt and response exchanges), propose initial codes for said episodes, and summarize observation logs. With the initial chat log codes, the research team then met again to review the initial themes from the chat log. As we met as a research team we then pivoted to reengaging with the facilitator observation notes to build thematic connections across coded data artifacts. We developed observation summaries for each session.

Another member of the research team who was not involved in the initial rounds of coding, completed a round of deductive coding using the observation summaries, chat log, exit interviews, screen recordings, and thematic insights of our first round of coding. We report on the most compelling findings from our inquiries.

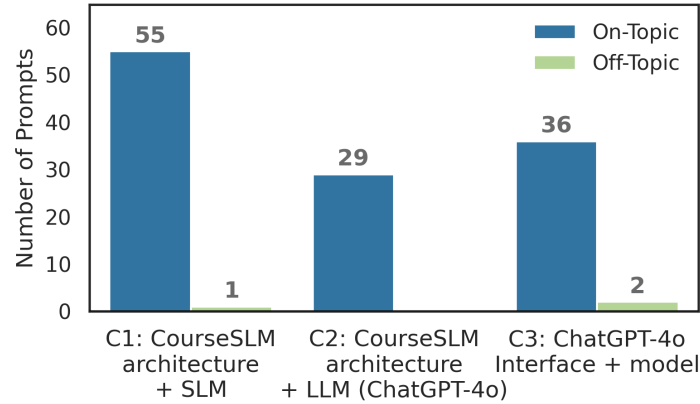


Fig. 8. Quantity of on-topic and off-topic prompt per session

5 Results

We present our findings in four parts. First, we analyze how CourseSLM and general-purpose models influenced participant focus, including the role of the classifier architecture and the impact of participant expectations. Second, we examine how participants addressed technical barriers using the AI tools across three conditions (see Fig. 8). Third, we detail participant perceptions of how AI tool use encouraged or discouraged learning momentum. Finally, we examine task performance across models and conditions, focusing on differences in code completion and how participants balanced AI support with curriculum materials and instructor assistance, reflecting on the implications for usability and future classroom integration.

5.1 RQ1: How do AI tools like CourseSLM and ChatGPT-4o support student task focus?

We use our classifier to identify off-topic messages showing 5% off-topic messages in the baseline C3 ChatGPT interface and model condition, 0% in C2 CourseSLM architecture with ChatGPT backend, and 1.8% in C1 CourseSLM architecture with SLM backend (see Fig. 8). Across all three studies the prevalence of off-task behavior was low to begin with, making it challenging to evaluate quantitatively. This may have been due to the study environment in which students were working on individual laptops at separate desks, making some feel like they were taking a standardized exam. We investigate the specific cases of off-topic behavior in our qualitative analysis below.

5.1.1 Condition 3: ChatGPT Baseline. We first analyze the baseline ChatGPT case which does not attempt to refocus students. We find overall that participants were left to their own devices. For example, throughout the majority of the session, Participant 15, used the tool to answer specific coding questions or tasks. However, near the end of the timed session, he turned to asking ChatGPT about cross country. ChatGPT made no attempts to refocus the user back to the given carbon footprint coding task.

"nxn" - **Participant 15**

"... It sounds like you might be referring to: - A math matrix (like $n \times n$ grid)? - A coding problem? - A game or tournament reference (like first champion)? ..." - **ChatGPT Baseline**

"nike cross n" - **Participant 15**

“Ahh — got it now! You’re talking about Nike Cross Nationals (NXN)... Nike Team Nationals (NTN) in 2004. - It features the top high school cross-country teams and individuals in the U.S. - It’s held every December at Glendoveer Golf Course in Portland, Oregon. Let me know if you want historical results, individual champs, or want to graph NXN data in Python” - **ChatGPT Baseline**

This study also highlighted how incorrect or hard-to-interpret AI responses could reduce student focus. When asked to describe his balance between using the AI tool and seeking help from the instructor, Participant 15 responded:

“I think it was probably 100% GPT” - **Participant 15**

This suggests that students relying heavily on AI to generate solutions may become derailed from making progress. Participant 15, a heavy user of the AI tool in condition 3, had the second most number of prompts (15 queries). With weekly ChatGPT exposure and experience coding for over a year (Table 1), he still had a comparable task completion percentage to the majority of participants in the study (27%, see Fig. 9). In comparison, participant 19 had the second fewest prompts (1 query) and was of similar age and coding/LLM experience to Participant 15, yet he completed the same number of exercises in less time (17 minutes vs 34 minutes, see Fig. 10). From the participants’ time lines, we can infer that Participant 15’s reliance of AI derailed and slowed progress.

5.1.2 Condition 2: CourseSLM Architecture and ChatGPT Backend. There were no instances of off topic queries to evaluate in this condition.

5.1.3 Condition 1: CourseSLM Architecture and Fine-tuned SLM Backend. In the few instances of off-topic queries, we find that CourseSLM successfully detects this and tries to redirect users. For example when Participant 3 sent the teasing prompt of “poop”, CourseSLM employed measured but clear language to recenter the conversation on the course curriculum.

“poop” - **Participant 3**

“I’m sorry, but that doesn’t look like a valid cell or question. Can you please clarify or rephrase your question?” - **CourseSLM**

Participant 3 then returned to completing the activity independently and sought task-related assistance from the AI tool. Knowing the tool would not give off topic responses, upon losing interest in the lesson again she instead turned to drawing on provided scratch paper. This example highlights the limit of any tool, showing that users who are not interested in the material can find ways to go off task outside of it.

Similar to the ChatGPT case above, we again find that a hallucination caused Participant 7 to lose trust, and then task focus. As a frequent general-purpose LLM user with no coding experience, he tried to drag a screenshot into the CourseSLM text input as shown in Fig. 11A. The tool is unable to process images, but instead returned an irrelevant response to the file name which was copied into the text box. This unfamiliar workflow and irrelevant response led to feelings of distrust with the tool. In an exit interview, Participant 7 constantly referred to the tool as “useless”, spending the rest of the activity time reading a background resource on data activism rather than completing the activity. Participant 7 enjoys learning from LLMs generally, claiming to struggle with focus and feeling like LLMs can generate personalized plans and parameters for executing tasks.

“Well, I definitely tried using the chatbot more at first because, with using ChatGPT, it’s like, more interactive. I can figure things out and it can give me some understanding... I came in with the assumption that this was like kind of a solo thing.” - **Participant 7**

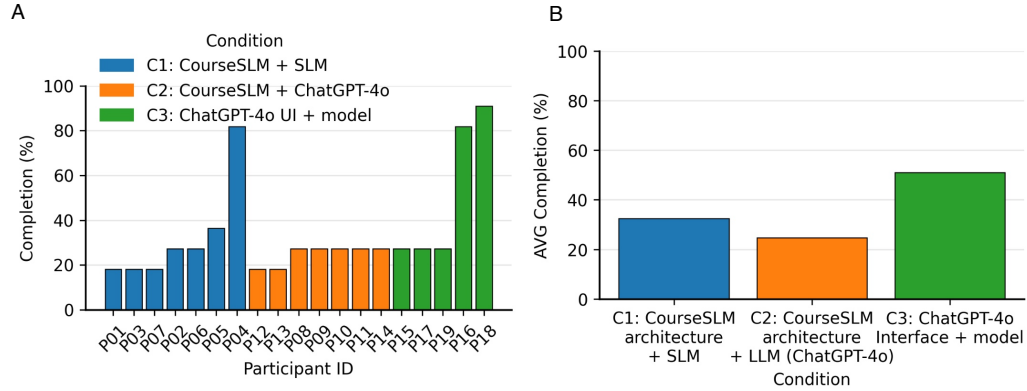


Fig. 9. **A) Task Completion per Participant** — Bars show the percentage of assigned tasks each participant completed; bar color indicates condition (C1, C2, C3). **B) Mean Task Completion by Condition** — Mean completion rate aggregated within condition.

This highlights the importance of educating users on how to appropriately use a tool like CourseSLM, with features designed to maintain focus and scaffold educational support, which may differ from conventional LLM workflows.

5.2 RQ2: How do AI tools like CourseSLM and ChatGPT-4o help students overcome technical barriers?

We evaluate whether each AI tool contributes to the participant triaging and resolving issues stalling their task progress. We find that the participants' coding experience often determined how effectively their tool use helped them overcome technical barriers. Even for frequent AI users, their lack of conceptual knowledge made it difficult to precisely explain their issue and appropriately interpret technical language across all tool responses. As explained below, we find that although Fig 9 suggests the ChatGPT condition C3 performed best, this is because students were directly copying full solutions provided by the chatbot, with little evidence of meaningful learning.

5.2.1 Condition 3: ChatGPT Baseline. Participants were most comfortable triaging their issues with the baseline ChatGPT. For novice programmers, the tool was **most useful in compensating for knowledge gaps**, with participants often copying and pasting entire code segments into their notebooks. Participant 18, for example, quickly generated Python code to compensate for his lack of familiarity with Python syntax.

“Well, I’m barely knowledgeable with Python. Just don’t use it as a main language. So that’s why it was just like kind of getting back into how did I use this language in general. And with the help of LLM, I feel like [it’s] a fairly easy thing to do because they give you a completed code segment” - **Participant 18**

However, when students used model generated solutions, they often moved on with executable code without realizing it didn’t fulfill the task requirements. For example, Participant 18 used ChatGPT written code to successfully display a bar chart, he did not use the specific colors outlined in the notebook instructions. Inexperienced users also had fundamental conceptual misunderstandings. For example, Participant 15 was instructed by the tool to import the pandas library, even though it was already imported in a hidden cell and the notebook instructions explained this. Rather than pausing to review the notebook instructions, he became frustrated when the tool didn’t solve the problem. Participant 15 also used

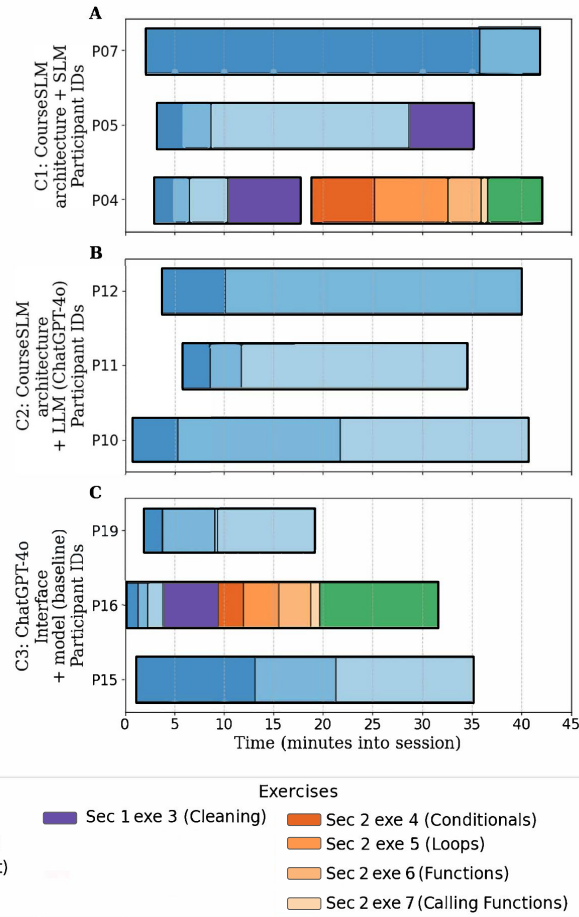


Fig. 10. Interviewed Participant task timelines by experimental condition. Colored bars show the duration spent on each task over 45 min. Panels: (A) Condition 1, (B) Condition 2, (C) Condition 3. Legend shorthand expands as follows: Sec = Section, exe = Exercise (e.g., “Sec 0 exe 0 (Vars)” = Section 0, Exercise 0: Variables; “Sec 0 exe 2 (Import)” = importing a CSV). Section 2 items include “exe 4 (Conditionals),” “exe 5 (Loops),” “exe 6 (Functions),” and “exe 7 (Calling Functions).” Visualization tasks are Sec 3

code with an incorrect file name. While one of the generalized debugging suggestions provided by the tool encouraged him to check the file name, a system with access to the course content like CourseSLM may have highlighted the oversight with explicit context.

“I would just write it and GPT give me a response like you probably did it right. Check this and that’d be fine.” - Participant 15

These examples illustrate that while ChatGPT can raise user scores on straightforward tasks, it does not foster deeper comprehension. Inexperienced students often stalled when encountering tasks or outputs they could not interpret. The promise of receiving an executable answer led participants to skim instructions rather than engage with them, causing some to assume they had overcome barriers without truly completing the task, or to spend additional time repairing errors that more thorough understanding could have prevented.

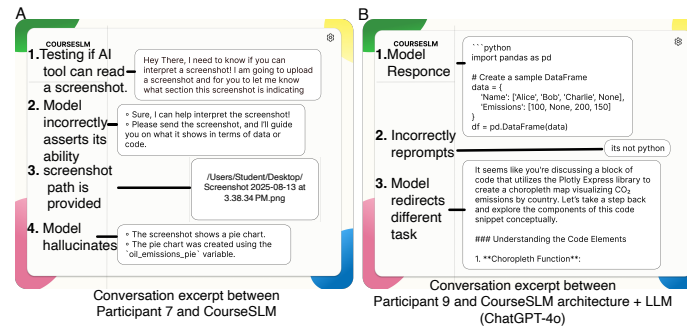


Fig. 11. **A) Conversation excerpt between Participant 7 and CourseSLM.** In this conversation, Participant 7 is testing to see if the AI tool can read a screenshot. The model incorrectly asserts that it can and after the screenshot is provided the model hallucinates a description of the image using course content. **B) Mock up of conversation log with Participant 9 and CourseSLM architecture + LLM (ChatGPT-4o).** The participant incorrectly re-prompts the model, and the model redirects the conversation to a different coding task.

5.2.2 Condition 2: CourseSLM Architecture and ChatGPT Backend. In this session participants often used the tool as a last resort because they wanted to solve problems on their own and found a lot of satisfaction from doing so. Once the tool was used, prompts with precise technical language helped resolve issues. Participants 12 and 14, for example, were able to fix a syntax error and gain a helpful clarification, respectively, using the tool.

“Whenever I got frustrated and couldn’t figure it out, I tried to go to the AI tool.” - **Participant 12**

In some cases participants became of frustrated due to long-winded AI responses that were difficult to parse and relate to their objectives. As **Participant 11** explained:

“...the response was like, it was just like really wordy. It gave me like a bunch of steps ... and it gave me information I didn’t need at the moment...” - **Participant 11**

When the AI tool and slideshow failed to provide succinct help, instructor support helped close knowledge gaps. As **Participant 11** describes it:

“Definitely asking for help from you guys and looking at the slideshow, if I didn’t like, understand something on a slideshow ... I would ask somebody for like, actual help and it was easier to understand.” - **Participant 11**

These examples highlight students’ interest in learning independently, but also underscore the need for teachers to close remaining knowledge gaps.

5.2.3 Condition 1: CourseSLM Architecture and Fine-tuned SLM Backend. For participants using the full CourseSLM, most were able to effectively overcome barriers through their use of the LLM. More experienced programmers such as Participant 4 used the AI tool to remind or clarify her of Python syntax. However, for less experienced participants, the tool could only help them until the task complexity surpassed their technical understanding.

Novice programmers like participants 9 and 15 used the suggested starting prompt “What is a variable?” to gain a conceptual understanding of their first task. As the tasks rose in difficulty, the participants expected to get executable code rather than stepwise high level explanations and hints provided by CourseSLM. Because the response was not what the participant was looking for, many struggled with re-prompting to get a different or more actionable output. In debriefing the activity, the participants agreed that CourseSLM accomplishes its goal of better answering “how to”

prompts to overcome barriers rather than “what to do” to get the right answer. While it was a point of frustration initially, some participants later appreciated and trusted CourseSLM even more for its tailored approach to support. As Participant 5 remarked:

“Honestly, it felt less like just kind of the AI generation and more like somebody had fed it this information and kind of taught it how to respond to specific tasks. So it felt a little bit more personal, which made me trust it a bit more.” - **Participant 5**

5.3 RQ3: How do AI tools like CourseSLM and ChatGPT-4o help students sustain learning momentum?

We explore whether the AI tools in this classroom setting supported learning momentum, defined by Robins’ learning edge momentum (LEM). The Learning Edge Momentum (LEM) concept suggests that for novice learners, early success or failure in acquiring introductory programming skills can set them on a trajectory of continued success or repeated failure [49]. Participants with 0–1 year of coding experience progressed through an average of 2.0 cells before stalling in C1, 2.29 cells in C2, and 3.0 cells in C3. Participants with 2–4 years of experience progressed through 2.5 cells before stalling in C1 and 9.0 cells in C3. We note there were no participants in C2 with 2–4 years of experience. These results again show that unmodified ChatGPT enables rapid progress for experienced users by providing solutions, but novice users stall when they do not understand AI responses.

5.3.1 Condition 3: ChatGPT Baseline. Participants in the baseline ChatGPT session seem to have adapted their long-term learning to be driven by AI. Across experience levels, participants directly copied and pasted the AI code snippets to quickly complete tasks, however many less experienced students stalled when they did not understand a response. Additionally, Participant 16 progressed to the second to last task before having a syntax issue. In her exit interview, she explains how she commonly relies on ChatGPT to develop working code for a research project due to her lack of formal computer science education.

“My project involves a lot of coding and I never actually took a coding class before, so... when I have a problem or a task that I need to complete, but I’m not sure about the specific, like lines of code or something, then I just use ChatGPT.” - **Participant 16**

Beyond our study, this quote suggests non-computing students may choose to skip formal computer science courses and instead use AI to achieve task-specific programming objectives.

Participant 15 even admitted, despite making task progress, when asked “what got in the way of your active learning during the study?” that ChatGPT was a hindrance:

“GPT I think, getting the answers, the answers for everything.” - **Participant 15**

5.3.2 Condition 2: CourseSLM Architecture and ChatGPT Backend. For the CourseSLM Architecture with ChatGPT condition, less experienced participants were also encouraged to learn more by using the tool. While the tool provided answers that less experienced programmers found useful, Participant 11 recognized she **needed stronger foundational knowledge to use AI appropriately** for future learning.

“I guess when I said like, ‘how do you do something?’ It kind of gave me like an answer like at the bottom. But I just didn’t know how to like interpret the answer because I don’t like, I really know what to do.” - **Participant 11**

5.3.3 *Condition 1: CourseSLM Architecture and Fine-tuned SLM Backend.* Most participants had success with conceptual prompting. However, when participants weren't getting concise answers, Participant 5 for example, spent time reprompting. He gained useful knowledge to help him complete the activity but found growth in his prompting strategy rather than curriculum knowledge. The progress Participant 5 made in the activity with CourseSLM gave him the confidence to continue learning more but he acknowledges his learning was not comprehensive.

"I would say I'm definitely more confident than I was before. I mean, I learned, like, a lot of new things doing this exercise... I wouldn't say, like, I have a good knowledge of it, but I definitely could see myself, like, continuing to do activities like this." - **Participant 5**

The deeper knowledge participants gained from the slides allowed them to prompt with more precision and also confirm the accuracy of the AI response, especially for the CourseSLM tool.

"Like, everything made sense to me, and it, like, matched up with, like, the explanations that were in the slides, too." - **Participant 5**

Participant 4, who is a more experienced Python programmer, used the AI tool to remind her of Python coding concepts. She assumed a strong stance against using AI. She asserts AI negatively impacts her learning progress and the environment. To address her concerns with AI use, she has a preference for seeking support from her in-person learning community. She finds the easy answers AI provides are not in line with her preferred learning process, which values self-efficacy and personal effort.

"I prefer to not use AI because I feel like it's detractive to my learning and especially in a classroom setting. I prefer to ask a professor or a TA or somebody I know in the class because, well, then it's just more personal. And then it's like you have these connections with people. And then also because I tend to understand it better, kind of what I mentioned before, like, I just learn it so much better. I'm not just given the answer. I have to work for it a little bit, and so I prefer that. Also, I've been learning a lot more about the environmental impact of AI, so I also try to avoid using it for that reason." - **Participant 4**

These examples highlight both the importance of pairing tools like CourseSLM with traditional techniques like slides and human instruction, and the opportunity for AI to support learning momentum for some students while a single instructor provides dedicated support to others.

5.4 RQ4: How do AI tools like CourseSLM and ChatGPT-4o affect task performance and completion?

Finally, we evaluate how the AI tools affected task performance and completion. We characterize task performance as the number of executable code cells written by participants, how comprehensively they fulfilled the task instructions, and the time spent on each task. Again, the intersection of their programming experience and AI experience greatly impacted the results.

Timeline analysis showed that participants in condition C1 completed a median of 3.57% of notebook tasks, compared to 2.71% in C2, and 5.60% in the baseline C3 condition (see Fig. 9B). Although averages were skewed by a few highly experienced individuals, overall completion was notably higher in C1 with the fine-tuned SLM than in C2 with the LLM backend. Because both C1 and C2 relied on the same CourseSLM architecture, these results suggest that larger general-purpose models are not inherently more effective when they cannot provide complete code solutions.

Across all conditions, participants were most successful across experience levels when they used a combination of all the resources provided to them. Each resource also offered unique benefits toward completing the task at hand. The

AI tool offered speed and allowed students to quickly execute code. However, when participants relied on AI alone, they often took shortcuts and did not fully meet the requirements from the given tasks.

5.4.1 Condition 3: ChatGPT Baseline. In this session some participants relied completely on AI generated code while others only using AI for debugging. For example, Participant 15, an inexperienced programmer who uses Chat GPT weekly, attributed in his words “probably 100%” of his task performance to the AI tool.

“I think. I think having GPT helped a lot. Yeah, I think that was pretty much it. I was essentially guessing because I didn’t really know what I was coding in. And then GPT was just like, let’s go with Python, essentially. And I just kind of rolled with it and it happened to work out.” - **Participant 15**

On the other hand, Participant 19 had a lot of coding and AI experience but does not attribute most of his performance to the AI tool. He rather limits his AI use to debugging.

“I more prefer to use it as like a nitpick, like a debugging kind of tool as opposed to like an instruction.” - **Participant 19**

Other experienced programmers in this session often fell in between these poles of AI reliance. They consistently used AI to generate code blocks they could apply based on their coding knowledge and understanding of the instructions.

5.4.2 Condition 2: CourseSLM Architecture and ChatGPT Backend. For the CourseSLM architecture with ChatGPT condition, many participants were focused on prompting efficiency to get most out of each response towards task progress. For example, Participant 10 describes how he prompted for the code of generalized programming concepts so he could apply the code across multiple contexts.

“So what I would do with AI, instead of trying to get a direct answer, I would try to figure out how to do something so I can use it in multiple situations instead of just one.” - **Participant 10**

Participants also directly pasted their code, errors, and task instructions to proceed through the tasks. Participants realized that in pasting these snippets, the model assumed they were more technically proficient and experienced than they were and provided very technically detailed answers users could not understand. In one case when Participant 9 received correct Python code from the tool as shown in Fig. 11, they asserted it was not Python. Rather than correcting the participant, the tool provided code for completely different problem causing frustration and preventing him from finishing the task.

5.4.3 Condition 1: CourseSLM Architecture and Fine-tuned SLM Backend. In the full CourseSLM session, Participant 7 was a frequent user of ChatGPT but had no programming experience before this activity. He made very little progress and attributed his poor task performance to the “uselessness” of the tool. He explained he often submits screenshots to ChatGPT to provide context to a prompt or in the case of this activity, explain what he should be doing for the given task. As described in the system design section, our development team intentionally did not build CourseSLM to be a multimodal to simplify user interactions. When his prompting strategy failed, he decided not to adapt his prompting strategy and abandon tool use. Given that he prefers using an AI tool, he made very little progress because he was anxious to seek the instructor help he needed.

“I don’t know what I’m doing. And then I would say a little self conscious with peers because a lot of the people in the class, I did know. And I would say part of me was like, I don’t know if I can ask these one on one questions or I felt like maybe a little embarrassed to.” - **Participant 7**

This again shows the need to introduce users to novel workflows.

6 Design & Development Implications for Community-based Computer Science Education AI Tools

In our exploratory user study of CourseSLM, we refined our understanding of our design goals through the user experiences of our target student population. Our deductive metrics for evaluating CourseSLM performance against general-purpose models (task focus, overcoming technical barriers, learning momentum, and task performance) yielded insights to inform future iterations of CourseSLM. Additionally, our findings raise broader design implications for the AI in computer science research community.

6.1 The Potential and Pitfalls of Self-Driven AI Learning

Most participants appreciated self-paced, interactive support and reported pride when solving issues independently. The affordances of AI educational tools favor self-driven learning approaches, which many students found motivating because they could complete tasks on their own. However, this self-directed mode of engagement also reinforced hesitancy to reach out for help from instructors, a tendency documented in prior work [44]. This dynamic was particularly concerning for students with high AI experience but limited programming background. In these cases, participants often relied heavily on the tool to navigate tasks but struggled to interpret responses when technical language exceeded their understanding. Individual AI use can also enable social withdrawal, especially among youth [68], and may discourage collaborative help-seeking from peers or instructors [4]. As seen in the ChatGPT condition, this pattern left some students stuck when full code outputs were generated but comprehension lagged behind, highlighting the tension between independence and meaningful learning. A classroom tool should therefore support self-driven progress while nudging toward collaborative norms (e.g., prompts that encourage sharing errors or discussing strategies with peers).

6.2 Challenging Sycophancy in Educational AI

Sycophantic responses can harm learning momentum by reinforcing short-term desires over long-term goals [25, 33]. Historically, subservience-as-helpfulness also reinforces harmful social narratives [5]. In education, such dynamics risk inflating confidence without comprehension, exacerbating Dunning–Kruger effects [35]. There is ongoing research, to develop non-sycophantic personas to serve unique pedagogical needs [23, 50].

Performance expectations shaped how some participants engaged with the tool. Many entered the study anticipating that AI would provide ready-made code, reflecting a growing norm where students expect technology to simply solve problems for them. This expectation often undercut early focus, as students struggled to adjust when CourseSLM instead offered scaffolded hints rather than full solutions. By refusing to provide direct code and instead emphasizing stepwise guidance, CourseSLM reframes “helpfulness” as comprehension-oriented rather than answer-oriented. Although this initially frustrated some students, it encouraged productive struggle, reinforced course alignment, and supported a shift in mindset toward learning rather than solution-seeking. Students also noted that concise, plain-language steps supported momentum, while lengthy or jargon-heavy replies hindered it.

6.3 CourseSLM and Data Stewardship in CS Education

We designed CourseSLM to supplement, rather than replace, classroom instructors. Across conditions, participants turned to instructors when barriers persisted beyond ten minutes, and nearly all challenges were resolved within five minutes of intervention. These findings suggest that human instruction remains critical for resolving persistent obstacles, while AI tools can provide intermediate scaffolding. A system that aligns with instructor workflows, addresses

knowledge gaps, and offers telemetry on common coding pitfalls holds promise as an integrated element of the classroom technology stack.

CourseSLM benefited from a deliberately limited knowledge base, which produced precise and contextualized vocabulary. Participants reported greater trust when the model’s scope and resources were transparent, stewardship responsibilities were clear, and cross-referencing with classroom materials (e.g., slides) was facilitated. Prior work shows that community-based stewardship can increase agency and trust [58]. In our study, students were also empowered to provide feedback directly to developers, creating opportunities for collaborative assessment and reinforcing the expectation that problems would be addressed at the system level.

Additionally, recent HCI work underscores links between sovereignty, stewardship, and educational AI [39, 41]. Enabling deployment practicality, privacy inference, prompt logs, and curricular RAG bundles locally supports stewardship while enabling offline operation. On-device storage supported our data stewardship goals by safeguarding student privacy, while enabling approved instructors, with student consent, to access prompt logs and related data to document classroom progress.

6.4 Actionable Improvements

Make Scope Legible. Before coding tasks, clearly communicating CourseSLM’s capabilities and limitations can help set accurate expectations. Streamlining these explanations into concise rationales reduces scanning costs for students, providing another concrete avenue for improving usability.

Adjust scaffold shape. We aim to structure responses in a consistent progression: beginning with a conceptual explanation, followed by an identifiable course-specific pattern, and concluding with sample code snippets.

Harden local-first pipelines. To support classroom deployment, we plan to extend local access pipelines that allow only approved instructors in close proximity to a student to gather limited data from the tool. This would enable instructors to fine-tune models, push updates to students’ AI systems, and access progress insights, ensuring support while maintaining data privacy.

Design for community, not isolation. We intend to seed prompts that invite peer discussion and instructor checkpoints, and offer “share this step” affordances by default, allowing collaborative prompting and resource distribution.

7 Limitations

7.1 Study Environment Limitations

A majority of study participants could only allocate 1.5 hours to user studies which limited our ability to provide an instructional lesson on data activism and introductory coding concepts. We intended to replicate the environment of a STEM-based classroom, but were unable to provide instruction then transition into the coding tasks. This may have had an impact on students’ comfortability during the study, and contributed to the unintended standardized test-taking atmosphere.

Observed focus gains are bounded by a short activity window (45 minutes) and introductory content. We cannot claim durability across longitudinal learning or advanced topics. For future studies with a similar one-session structure, we would like to examine student performance given unspecified time limitations, where studies conclude once the student completes each coding task or resigns.

Migration to other devices is not yet possible. Our tool works on Apple M-series hardware. While a Python server reduced variability and network dependence during study deployment, implementations for other hardware and operating systems are necessary extensions for future work.

7.2 Empirical Limitations

Internal validity. A single participant discovered a “jailbreak” path by pasting code comments from study materials, momentarily breaching the scaffolded persona. This is another area of improvement that we will address before facilitating additional user studies. **Construct validity.** “Stuckness,” “momentum,” and “focus” were inferred from logs and observation; future studies should triangulate with IDE/keystroke traces and pre-register verbosity/readability as measures. **External validity.** Findings center around young programming novices and introductory Python data-visualization content; generalization to other populations, topics, or longer time horizons remains uncharted. **Conclusion validity.** Sample size and time-on-task limit sensitivity to small between-condition effects; both local and cloud models sometimes produced long, technical responses, complicating out understanding of momentum differences.

8 Conclusion

CourseSLM shows that locally run, fine-tuned SLMs combined with a focus-oriented architecture can meaningfully support student learning in culturally grounded computing curricula. By constraining outputs to lesson-specific assistance and redirecting off-task queries, the system helped students stay focused while preserving privacy and equity through on-device execution. Across three classroom conditions, the CourseSLM architecture showed support in maintaining student focus, the SLM helped overcome technical barriers and sustained learning momentum, and overall the SLM performed comparably to general-purpose LLMs all without providing full code solutions. Although some students with prior AI experience were initially frustrated by refusals to supply complete answers, scaffolded hints encouraged comprehension and reframed expectations of helpfulness toward learning rather than solution-seeking. Participants also valued the transparency of a limited knowledge base and trusted the tool more when scope and stewardship were clear, particularly when aligned with community-responsive curricula and opportunities for feedback. Although our evaluation centered on data activism and introductory programming, the broader framework of local execution and equity-centered design extends to diverse educational contexts. This work highlights the potential for local-first educational AI tools to foster student engagement, learning, and data stewardship while complementing, rather than replacing, teacher instruction.

References

- [1] Mohammad Abolnejadian, Sharareh Alipour, and Kamyar Taeb. 2024. Leveraging ChatGPT for Adaptive Learning through Personalized Prompt-based Instruction: A CS1 Education Case Study. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (*CHI EA '24*). Association for Computing Machinery, New York, NY, USA, Article 521, 8 pages. doi:10.1145/3613905.3637148
- [2] Inc. Apple. [n. d.]. Apple Open Source — opensource.apple.com. <https://opensource.apple.com/projects/mlx/>. [Accessed 06-09-2025].
- [3] R. J. Barker and E. J. Unger. 1983. A predictor for success in an introductory programming class based upon abstract reasoning development. *ACM SIGCSE Bulletin* 15, 1 (1983), 154–158. doi:10.1145/800038.801037
- [4] Patrick Bassner, Eduard Frankford, and Stephan Krusche. 2024. Iris: An AI-Driven Virtual Tutor for Computer Science Education. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1* (Milan, Italy) (*ITiCSE 2024*). Association for Computing Machinery, New York, NY, USA, 394–400. doi:10.1145/3649217.3653543
- [5] Ruha Benjamin. 2019. *Race After Technology: Abolitionist Tools for the New Jim Code*. Polity, USA.
- [6] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. arXiv:<https://doi.org/10.1191/1478088706qp0630a> doi:10.1191/1478088706qp0630a

- [7] Neil C. C. Brown, Pierre Weill-Tessier, Juho Leinonen, Paul Denny, and Michael Kölling. 2025. Howzat? Appealing to Expert Judgement for Evaluating Human and AI Next-Step Hints for Novice Programmers. *ACM Trans. Comput. Educ.* 25, 3, Article 31 (July 2025), 43 pages. doi:10.1145/3737885
- [8] Jun Ho Choi, Oliver Garrod, Paul Atherton, Andrew Joyce-Gibbons, Miriam Mason-Sesay, and Daniel Björkegren. 2024. Are LLMs Useful in the Poorest Schools? TheTeacher.AI in Sierra Leone. arXiv:2310.02982 [cs.CY] <https://arxiv.org/abs/2310.02982>
- [9] Aisha Cora, Vicente Arroyos, Liban Hussein, Vikram Iyer, R. Benjamin Shapiro, and Kyle Johnson. 2024. Leveraging AI to Improve STEM Engagement for Black and Latine Youth. In *2024 Black Issues in Computing Education (BICE)*. xxiii–xxiii. doi:10.1109/BICE60192.2024.00031
- [10] Tyne Crow, Andrew Luxton-Reilly, and Burkhard Wuensche. 2018. Intelligent tutoring systems for programming education: a systematic review (*ACE '18*). Association for Computing Machinery, New York, NY, USA, 53–62. doi:10.1145/3160489.3160492
- [11] Ali Darvishi, Hassan Khosravi, Shazia Sadiq, Dragan Gašević, and George Siemens. 2024. Impact of AI assistance on student agency. *Computers & Education* 210 (2024), 104967. doi:10.1016/j.compedu.2023.104967
- [12] Badhan Chandra Das, M. Hadi Amini, and Yanzhao Wu. 2025. Security and Privacy Challenges of Large Language Models: A Survey. *ACM Comput. Surv.* 57, 6, Article 152 (Feb. 2025), 39 pages. doi:10.1145/3712001
- [13] Michael de Raadt, Margaret Hamilton, Raymond F. Lister, Jodi Tutty, Bob Baker, Ilona Box, Quintin I. Cutts, Sally Fincher, John Hamer, Patricia Haden, and Marian Petre. 2005. Approaches to Learning in Computer Programming Students and Their Effect on Success. In *Proceedings of the 2005 Annual Conference of the Higher Education Research and Development Society of Australasia (HERDSA)*, Vol. 28. Higher Education Research and Development Society of Australasia, Milperra, NSW, Australia, 407–414.
- [14] Zachary Englhardt, Richard Li, Dilini Nissanka, Zhihan Zhang, Girish Narayanswamy, Joseph Breda, Xin Liu, Shwetak Patel, and Vikram Iyer. 2024. Exploring and Characterizing Large Language Models for Embedded System Development and Debugging. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI EA '24)*. Association for Computing Machinery, New York, NY, USA, Article 150, 9 pages. doi:10.1145/3613905.3650764
- [15] Zachary Englhardt, Chengqian Ma, Margaret E Morris, Chun-Cheng Chang, Xuhai" Orson" Xu, Lianhui Qin, Daniel McDuff, Xin Liu, Shwetak Patel, and Vikram Iyer. 2024. From classification to clinical insights: Towards analyzing and reasoning about mobile and behavioral health data with large language models. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8, 2 (2024), 1–25.
- [16] Hugging Face. 2021. sentence-transformers/all-MiniLM-L6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. Sentence embedding model (384-dim), Apache-2.0 License.
- [17] Stephen T. Fife and Jacob D. Gossner. 2024. Deductive Qualitative Analysis: Evaluating, Expanding, and Refining Theory. *International Journal of Qualitative Methods* 23 (2024), 16094069241244856. arXiv:<https://doi.org/10.1177/16094069241244856> doi:10.1177/16094069241244856
- [18] Anonymized for Review. 2025. In press..
- [19] Eduard Frankford, Clemens Sauerwein, Patrick Bassner, Stephan Krusche, and Ruth Breu. 2024. AI-Tutoring in Software Engineering Education. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training (Lisbon, Portugal) (ICSE-SEET '24)*. Association for Computing Machinery, New York, NY, USA, 309–319. doi:10.1145/3639474.3640061
- [20] Diana Franklin, Paul Denny, David A. Gonzalez-Maldonado, and Minh Tran. 2025. *Generative AI in Computer Science Education: Challenges and Opportunities*. Cambridge University Press.
- [21] Catherine A. Gao, Frederick M. Howard, Nikolay S. Markov, Emma C. Dyer, Siddhi Ramesh, Yuan Luo, and Alexander T. Pearson. 2023. Comparing scientific abstracts generated by ChatGPT to real abstracts with detectors and blinded human reviewers. *NPJ Digital Medicine* 6, 1 (2023), 75. doi:10.1038/s41746-023-00819-6
- [22] Geneva Gay. 2002. Preparing for Culturally Responsive Teaching. *Journal of Teacher Education* 53, 2 (2002), 106–116. doi:10.1177/0022487102053002003
- [23] Quinn Goddard, Nathan Moton, Jonathan Hudson, and Helen Ai He. 2024. A Chatbot Won't Judge Me: An Exploratory Study of Self-disclosing Chatbots in Introductory Computer Science Classes. In *Proceedings of the 26th Western Canadian Conference on Computing Education (Kelowna, BC, Canada) (WCCCE '24)*. Association for Computing Machinery, New York, NY, USA, Article 9, 7 pages. doi:10.1145/3660650.3660662
- [24] Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang, Andy Narayanan, Aonan Zhang, Bowen Zhang, Chen Chen, Chung-Cheng Chiu, David Qiu, Deepak Gopinath, Dian Ang Yap, Dong Yin, Feng Nan, Floris Weers, Guoli Yin, Haoshuo Huang, Jianyu Wang, Jiarui Lu, John Peebles, Ke Ye, Mark Lee, Nan Du, Qibin Chen, Quentin Keunebroek, Sam Wiseman, Syd Evans, Tao Lei, Vivek Rathod, Xiang Kong, Xianzhi Du, Yanghao Li, Yongqiang Wang, Yuan Gao, Zaid Ahmed, Zhaoyang Xu, Zhiyun Lu, Al Rashid, Albin Madappally Jose, Alec Doane, Alfredo Bencomo, Allison Vanderby, Andrew Hansen, Ankur Jain, Anupama Mann Anupama, Areeba Kamal, Bugu Wu, Carolina Brum, Charlie Maalouf, Chinguun Erdenebileg, Chris Dulhanty, Dominik Moritz, Doug Kang, Eduardo Jimenez, Evan Ladd, Fangping Shi, Felix Bai, Frank Chu, Fred Hohman, Hadas Koteck, Hannah Gillis Coleman, Jane Li, Jeffrey Bigham, Jeffery Cao, Jeff Lai, Jessica Cheung, Jiulong Shan, Joe Zhou, John Li, Jun Qin, Karanjeet Singh, Karla Vega, Kelvin Zou, Laura Heckman, Lauren Gardiner, Margit Bowler, Maria Cordell, Meng Cao, Nicole Hay, Nilesh Shahdarpuri, Otto Godwin, Pranay Dighe, Pushyami Rachapudi, Ramsey Tantawi, Roman Frigg, Sam Davarnia, Sanskruti Shah, Saptarshi Guha, Sasha Sirovica, Shen Ma, Shuang Ma, Simon Wang, Sulgi Kim, Suma Jayaram, Vaishaal Shankar, Varsha Paidi, Vivek Kumar, Xin Wang, Xin Zheng, Walker Cheng, Yael Shrager, Yang Ye, Yasu Tanaka, Yihao Guo, Yunsong Meng, Zhao Tang Luo, Zhi Ouyang, Alp Aygar, Alvin Wan, Andrew Walkingshaw, Andy Narayanan, Antonie Lin, Arsalan Farooq, Brent Ramerth, Colorado Reed, Chris Bartels, Chris Chaney, David Riazati, Eric Liang Yang, Erin Feldman, Gabriel Hochstrasser, Guillaume Seguin, Irina Belousova, Joris Pelemans, Karen Yang, Keivan Alizadeh Vahid, Liangliang Cao, Mahyar Najibi, Marco Zuliani, Max Horton, Minsik Cho, Nikhil Bhendawade, Patrick Dong, Piotr Maj, Pulkit Agrawal, Qi Shan, Qichen Fu, Regan Poston, Sam Xu, Shuangning Liu, Sushma Rao, Tashweena Heeramun, Thomas Merth, Uday Rayala, Victor Cui, Vivek Rangarajan Sridhar, Wencong Zhang, Wenqi Zhang, Wentao Wu, Xingyu Zhou, Xinwen Liu, Yang Zhao, Yin Xia, Zhile Ren, and Zhongzheng Ren. 2024. Apple Intelligence Foundation Language

- Models. arXiv:2407.21075 [cs.AI] <https://arxiv.org/abs/2407.21075>
- [25] Emily Herron, Junqi Yin, and Feiyi Wang. 2024. SciTrust: Evaluating the Trustworthiness of Large Language Models for Science. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 72–78. doi:10.1109/SCW63240.2024.00017
- [26] Hugging Face Transformers Team. 2024. Chat templates. https://huggingface.co/docs/transformers/en/chat_templating. Accessed 2025-09-11.
- [27] Hugging Face Transformers Team. 2024. Writing a chat template. https://huggingface.co/docs/transformers/en/chat_templating_writing. Accessed 2025-09-11.
- [28] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [29] Anubhav Jangra, Jamshid Mozafari, Adam Jatowt, and Smaranda Muresan. 2025. Navigating the Landscape of Hint Generation Research: From the Past to the Future. *Transactions of the Association for Computational Linguistics* 13 (06 2025), 505–528. doi:10.1162/tacl_a_00751
- [30] Kyle Johnson, Vicente Arroyos, Celeste Garcia, Liban Hussein, Aisha Cora, Tsewone Melaku, Jay L. Cunningham, R. Benjamin Shapiro, and Vikram Iyer. 2024. AVELA - A Vision for Engineering Literacy & Access: Understanding Why Technology Alone Is Not Enough. arXiv:2401.14581 [cs.CY] <https://arxiv.org/abs/2401.14581>
- [31] Kyle Johnson, Vicente Arroyos, Liban Hussein, Aisha Cora, Eman Sherif, Celeste Garcia, Teanna Barrett, Saara Uthmaan, Samira Shirazy, Jay Cunningham, R. Benjamin Shapiro, and Vikram Iyer. 2024. Scalable Community Mentorship: A Vision for Engineering Literacy & Access. In *2024 World Engineering Education Forum - Global Engineering Deans Council (WEEF-GEDC)*. 1–9. doi:10.1109/WEEF-GEDC63419.2024.10854949
- [32] Enkelejda Kasneci, Kathrin Sessler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Jürgen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, Jochen Weller, Jochen Kuhn, and Gjergji Kasneci. 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences* 103 (2023), 102274. doi:10.1016/j.lindif.2023.102274
- [33] Azal Ahmad Khan, Sayan Alam, Xinran Wang, Ahmad Faraz Khan, Debanga Raj Neog, and Ali Anwar. 2024. Mitigating Sycophancy in Large Language Models via Direct Preference Optimization. In *2024 IEEE International Conference on Big Data (BigData)*. 1664–1671. doi:10.1109/BigData62323.2024.10825538
- [34] Nataliya Kosmyrna, Eugene Hauptmann, Ye Tong Yuan, Jessica Situ, Xian-Hao Liao, Ashly Vivian Beresnitzky, Iris Braunstein, and Pattie Maes. 2025. Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task. arXiv:2506.08872 [cs.AI] <https://arxiv.org/abs/2506.08872>
- [35] Justin Kruger and David Dunning. 1999. Unskilled and unaware of it: how difficulties in recognizing one’s own incompetence lead to inflated self-assessments. *Journal of personality and social psychology* 77, 6 (1999), 1121.
- [36] Gloria Ladson-Billings. 1995. Toward a Theory of Culturally Relevant Pedagogy. *American Educational Research Journal* 32, 3 (1995), 465–491. <http://www.jstor.org/stable/1163320>
- [37] JA Large and Jamshid Beheshti. 2005. Interface design, web portals, and children. *Library Trends* 54, 2 (2005), 318–342.
- [38] Mengqi Liu and Faten M’Hiri. 2024. Beyond Traditional Teaching: Large Language Models as Simulated Teaching Assistants in Computer Science. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, OR, USA) (*SIGCSE 2024*). Association for Computing Machinery, New York, NY, USA, 743–749. doi:10.1145/3626252.3630789
- [39] Joy Ming, Hawi H Tolera, Jiamin Tu, Ella Yitzhaki, Chit Sum Eunice Ngai, Madeline Sterling, Ariel C Avgar, Aditya Vashistha, and Nicola Dell. 2025. Exploring Data-Driven Advocacy in Home Health Care Work. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI ’25)*. Association for Computing Machinery, New York, NY, USA, Article 97, 17 pages. doi:10.1145/3706598.3713086
- [40] Luis Morales-Navarro, Yasmin B. Kafai, Francisco E. Castro, William Payne, Kayla DesPortes, Daniella DiPaola, Randi Williams, Safinah Ali, Cynthia Breazeal, Clifford Lee, Elisabeth Soep, Duri Long, Brian Magerko, Jaemarie Solyst, Amy Ogan, Cansu Tatar, Shiyan Jiang, Jie Chao, Carolyn P. Rosé, and Sepehr Vakil. 2023. Making Sense of Machine Learning: Integrating Youth’s Conceptual, Creative, and Critical Understandings of AI. *arXiv preprint arXiv:2305.02840* (2023). <https://arxiv.org/abs/2305.02840>
- [41] Sukanya Kannan Moudgalya and Sai Swaminathan. 2024. Toward Data Sovereignty: Justice-oriented and Community-based AI Education. In *Proceedings of the 2024 on RESPECT Annual Conference* (Atlanta, GA, USA) (*RESPECT 2024*). Association for Computing Machinery, New York, NY, USA, 94–99. doi:10.1145/3653666.3656107
- [42] Judith S Olson and Wendy A Kellogg. 2014. *Ways of Knowing in HCI*. Vol. 2. Springer.
- [43] Google PAIR. 2025. People + AI Guidebook — pair.withgoogle.com/guidebook/.
- [44] Prajish Prasad and Aamod Sane. 2024. A Self-Regulated Learning Framework using Generative AI and its Application in CS Educational Intervention Design. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, OR, USA) (*SIGCSE 2024*). Association for Computing Machinery, New York, NY, USA, 1070–1076. doi:10.1145/3626252.3630828
- [45] James Prather, Brent N. Reeves, Paul Denny, Brett A. Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. “It’s Weird That it Knows What I Want”: Usability and Interactions with Copilot for Novice Programmers. *ACM Trans. Comput.-Hum. Interact.* 31, 1, Article 4 (Nov. 2023), 31 pages. doi:10.1145/3617367
- [46] Pallets Projects. 2025. *Jinja Documentation*. Pallets. Version 3.1.x. Accessed 2025-09-11.
- [47] Mohammad M.H. Raihan, Sujoy Subroto, Nashit Chowdhury, Katharina Koch, Erin Ruttan, and Tanvir C. Turin. 2024. Dimensions and barriers for digital (in)equity and digital divide: a systematic integrative review. *Digital Transformation and Society* 4, 2 (08 2024), 111–127. arXiv:<https://www.emerald.com/dts/article-pdf/4/2/111/9718758/dts-04-2024-0054.pdf> doi:10.1108/DTS-04-2024-0054

- [48] Sheikh Faisal Rashid, Nghia Duong-Trung, and Niels Pinkwart. 2024. Generative AI in Education: Technical Foundations, Applications, and Challenges. In *Artificial Intelligence and Education*, Seifedine Kadry (Ed.). IntechOpen, Rijeka, Chapter 2. doi:10.5772/intechopen.1005402
- [49] Anthony Robins. 2010. Learning edge momentum: a new account of outcomes in CS1. *Computer Science Education* 20, 1 (2010), 37–71. arXiv:https://doi.org/10.1080/08993401003612167 doi:10.1080/08993401003612167
- [50] Kantwon Rogers, Michael Davis, Mallesh Maharana, Pete Etheredge, and Sonia Chernova. 2025. Playing Dumb to Get Smart: Creating and Evaluating an LLM-based Teachable Agent within University Computer Science Classes. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 126, 22 pages. doi:10.1145/3706598.3713644
- [51] Cynthia K. Sanders and Edward Scanlon. 2021. The Digital Divide Is a Human Rights Issue: Advancing Social Inclusion Through Social Work Advocacy. *Journal of Human Rights and Social Work* 6, 2 (2021), 2365–1792. doi:10.1007/s41134-020-00147-9
- [52] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint abs/1910.01108* (2019). <https://arxiv.org/abs/1910.01108>
- [53] Leen-Kiat Soh. 2006. Incorporating an intelligent tutoring system into CS1. In *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education* (Houston, Texas, USA) (*SIGCSE '06*). Association for Computing Machinery, New York, NY, USA, 486–490. doi:10.1145/1121341.1121494
- [54] Jaemarie Solyst, Cindy Peng, Wesley Hanwen Deng, Praneetha Pratapa, Jessica Hammer, Amy Ogan, Jason Hong, and Motahhare Eslami. 2025. Investigating Youth AI Auditing. *arXiv preprint arXiv:2502.18576* (2025). <https://arxiv.org/abs/2502.18576>
- [55] Jaemarie Solyst, Shixian Xie, Ellia Yang, Angela E.B. Stewart, Motahhare Eslami, Jessica Hammer, and Amy Ogan. 2023. I would Like to design": Black Girls Analyzing and Ideating Fair and Accountable AI. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*.
- [56] T. Subha and P. Jakulin Divya Mary. 2024. Integrating Large Language Models into Education: Revolutionising Teaching and Learning. *International Journal of Educational Sciences* 47, 3 (2024), 75–90. doi:10.31901/24566322.2024/47.03.1388
- [57] Mei Tan, Hansol Lee, Dakuo Wang, and Hari Subramonyam. 2024. Is a Seat at the Table Enough? Engaging Teachers and Students in Dataset Specification for ML in Education. *Proc. ACM Hum.-Comput. Interact.* 8, CSCW1, Article 81 (April 2024), 32 pages. doi:10.1145/3637358
- [58] Emily Tseng, Rosanna Bellini, Yeuk-Yu Lee, Alana Ramjit, Thomas Ristenpart, and Nicola Dell. 2024. Data Stewardship in Clinical Computer Security: Balancing Benefit and Burden in Participatory Systems. *Proc. ACM Hum.-Comput. Interact.* 8, CSCW1, Article 39 (April 2024), 29 pages. doi:10.1145/3637316
- [59] Raechel Walker. 2025. Liberatory Computing: Empowering African American Students Through Data Activism. In *Proceedings of the 2025 Conference on Research on Equitable and Sustained Participation in Engineering, Computing, and Technology* (Newark, NJ, USA) (*RESPECT 2025*). Association for Computing Machinery, New York, NY, USA, 387–388. doi:10.1145/3704637.3734750
- [60] Raechel Walker, Olivia Dias, Matthew Taylor, and Cynthia Breazeal. 2024. Alleviating the Danger Of A Single Story Through Liberatory Computing Education. In *Proceedings of the 2024 on RESPECT Annual Conference* (Atlanta, GA, USA) (*RESPECT 2024*). Association for Computing Machinery, New York, NY, USA, 169–178. doi:10.1145/3653666.3656072
- [61] Raechel Walker, Eman Sherif, and Cynthia Breazeal. 2022. Liberatory Computing Education for African American Students. In *2022 Conference on Research in Equitable and Sustained Participation in Engineering, Computing, and Technology (RESPECT)*. IEEE Computer Society, Los Alamitos, CA, USA, 95–99. doi:10.1109/RESPECT55273.2022.00024
- [62] Zijun Wu, Bingyuan Liu, Ran Yan, Lei Chen, and Thomas Delteil. 2024. Reducing Distraction in Long-Context Language Models by Focused Learning. arXiv:2411.05928 [cs.CL] <https://arxiv.org/abs/2411.05928>
- [63] Y. M. Yan, C. Q. Chen, Y. B. Hu, et al. 2025. LLM-based collaborative programming: impact on students' computational thinking and self-efficacy. *Humanities and Social Sciences Communications* 12, 1 (2025), 149. doi:10.1057/s41599-025-04471-1
- [64] An Yang et al. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024). <https://arxiv.org/abs/2407.10671>
- [65] J.D. Zamfirescu-Pereira, Laryn Qi, Björn Hartmann, John DeNero, and Narges Norouzi. 2025. 61A Bot Report: AI Assistants in CS1 Save Students Homework Time and Reduce Demands on Staff. (Now What?). In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1* (Pittsburgh, PA, USA) (*SIGCSE 2025*). Association for Computing Machinery, New York, NY, USA, 1309–1315. doi:10.1145/3641554.3701864
- [66] Chunpeng Zhai, Santoso Wibowo, and Lily D. Li. 2024. The effects of over-reliance on AI dialogue systems on students' cognitive abilities: a systematic review. *Smart Learning Environments* 11, 1 (2024), 28. doi:10.1186/s40561-024-00316-7
- [67] Chunyan Zhai, Stefanus Wibowo, and Li Da Li. 2024. The effects of over-reliance on AI dialogue systems on students' cognitive abilities: A systematic review. *Smart Learning Environments* 11, 1 (2024), 28. doi:10.1186/s40561-024-00316-7
- [68] Renwen Zhang, Han Li, Han Meng, Jinyuan Zhan, Hongyuan Gan, and Yi-Chieh Lee. 2025. The Dark Side of AI Companionship: A Taxonomy of Harmful Algorithmic Behaviors in Human-AI Relationships. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 13, 17 pages. doi:10.1145/3706598.3713429
- [69] Zhihan Zhang, Felix Hähnlein, Yuxuan Mei, Zachary Englhardt, Shwetak Patel, Adriana Schulz, and Vikram Iyer. 2024. DeltaLCA: Comparative Life-Cycle Assessment for Electronics Design. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8, 1 (2024), 1–29.
- [70] Meina Zhu and Ke Zhang. 2025. Artificial Intelligence for Computer Science Education in Higher Education: A Systematic Review of Empirical Research Published in 2003–2023: M. Zhu, K. Zhang. *Technology, Knowledge and Learning* (2025), 1–25.
- [71] Yihao Zhu, Zhoutong Ye, Yichen Yuan, Wenxuan Tang, Chun Yu, and Yuanchun Shi. 2025. AutoPBL: An LLM-powered Platform to Guide and Support Individual Learners Through Self Project-based Learning. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 584, 26 pages. doi:10.1145/3706598.3714261

A Participant Demographics

Table 1: Overview of Interviewed Student Demographics, ChatGPT Exposure, and Project Completion

ID	Age	Project Completed	Gender	Race/ Ethnicity	Grade Level	ChatGPT Exposure
1	20	No	Man	Black or African American	3rd Year in College	Weekly for the past 1+ year(s)
2	21	No	Woman	Black or African American	4th year in College	Daily for the past 1+ year(s)
3	17	No	Woman	Black or African American	4th Year in High School	Weekly for the past 1+ year(s)
4	20	No	Woman	White, Latinx	4th Year in College	Monthly for the past 4-6 months
5	21	No	Man	Black or African American	4th Year in College	Daily for the past 1+ year(s)
6	19	No	Woman	Black or African American	3rd Year in College	Monthly for the past 4-6 months
7	28	No	Man	Black or African American	4th Year in College	Weekly for the past 1+ year(s)
8	20	No	Man	Latinx	3rd Year in College	-
9	15	No	Man	Black or African American	2nd Year in High School	Monthly for the past 1 month
10	15	No	Man	Asian, White, Native American	2nd Year in High School	-
11	17	No	Woman	Black or African American	4th Year in High School	Monthly for the past 7-12 months
12	36	No	Woman	Black or African American	3rd Year in College	Monthly for the past 1+ year(s)
13	18	No	Man	Black or African American	1st Year in College	Monthly for the past 1-3 months
14	15	No	Woman	Black or African American	3rd Year in High School	Monthly for the past 1+ year(s)
15	17	No	Man	Black or African American	3rd Year in High School	Weekly for the past 1+ year(s)

Continued on next page

Table 1: Overview of Interviewed Student Demographics, ChatGPT Exposure, and Project Completion (Continued)

ID	Age	Project Completed	Gender	Race/ Ethnicity	Grade Level	ChatGPT Exposure
16	29	No	Woman	Asian	4th Year in College	Daily for the past 4-6 months
17	20	No	Woman	Black or African American	2nd Year in College	Daily for the past 4-6 months
18	20	Yes	Man	Asian	3rd Year in College	Weekly for the past 7-12 months
19	20	No	Man	Black or African American	3rd Year in College	Daily for the past 1+ year(s)

B Codebook

Table 2: Thematic codebook detailing the codes, descriptions, and illustrative examples used in the qualitative analysis of student interactions with the AI tool.

Theme	Description	Example
1 - Impact of AI Tool on Student Task Focus		
The use of AI tool during study impacts the participant focus on programming tasks		
11 - Distracting AI Feature or Performance	The participant is distracted from focusing on the task by using the AI tool.	"I was just trying to get a basic understanding of if it could even comprehend it. I was like, if it can't tell me that this is section zero, then it like confirmed that this is useless because like they can read my screenshot. This has nothing to do with that, right?"
12 - Self-Driven Approach Impact on AI Use	The participant's approach to self-driven learning impacts how they use the AI tool.	Observation Note: The participant spent a considerable amount of time drawing.
13 - Focusing AI Feature or Performance	The participant is directed to focus on the task by using the AI tool.	"Honestly, it felt less like just kind of the AI generation and more like somebody had fed it this information and kind of taught it how to respond to specific tasks. So it felt a little bit more personal, which made me trust it a bit more."
2 - Impact of AI Tool on Overcoming Technical Barriers		
AI tool is used to overcome technical barriers		
21 - Seeking Resources Outside of AI Tool	The participant seeks help for overcoming technical barriers from the slideshow or the instructor.	"I think it was the slides. Like, the slides helped a lot. I really didn't want to ask for help, mainly because I felt like, okay, if I can read this. And then whenever I got frustrated and couldn't figure it out, I tried to go to the AI tool."
22 - AI Fails to Support Technical Progress	The support the participant seeks from the AI tool is not fulfilled.	Chat prompt: "That didn't work"

Continued on next page

Table 2: Thematic codebook detailing the codes, descriptions, and illustrative examples used in the qualitative analysis of student interactions with the AI tool. (Continued)

Theme	Description	Example
23 - AI Succeeds in Supporting Technical Progress	The support the participant seeks from the AI tool is fulfilled.	“Because it was more like just like quick and it was like easy to like get your information. Just copy and paste it into the box and send it and then it would just respond to you.”
24 - Programming Experience Impact	The participant’s programming/computing experience impacts how they use the tool and its effectiveness.	“I guess when I said like, how do you do something? It kind of gave me like an answer like at the bottom. But I just didn’t know how to like interpret the answer because I don’t like, I really know what to do.”
3 - Impact of AI Tool on Sustaining Learning Momentum		
After using the AI tool, participant indicates deeper understanding and are encouraged to continue learning programming skills		
31 - Concerns with AI for Learning Momentum	The participant raises concerns they have with using AI to continue learning the curriculum.	“I would say I’m definitely more confident than I was before. I mean, I learned, like, a lot of new things doing this exercise... I wouldn’t say, like, I have a good knowledge of it, but I definitely could see myself, like, continuing to do activities like this.”
32 - Potential with AI Use for Learning Momentum	The participant highlights the potential for using AI to continue learning the curriculum.	“I could probably use it again if I was like, but let’s say, if I had the class and I had, like, I needed, like, a study guide or something, I could probably, like, put, like, the topics and themes of what I’m studying in there to be like, oh, make me a study guide.”
4 - Impact of AI Tool on Task Performance		
The participant uses the AI tool to comprehensively fulfill task instructions and progress through the given set of tasks.		

Continued on next page

Table 2: Thematic codebook detailing the codes, descriptions, and illustrative examples used in the qualitative analysis of student interactions with the AI tool. (Continued)

Theme	Description	Example
41 - AI Prompting Approaches	The participant develops or deploys an approach to prompting to support task performance.	“So what I would do with AI, instead of trying to get a direct answer, I would try to figure out how to do something so I can use it in multiple situations instead of just one.”
42 - Task Completeness	The completeness of the task by the participant is noted.	Observation note: “The participant was the only one to complete the full notebook.”